CLASE 4.1 - Sistemas Digitales

Semana 4: Programación y funcionamiento del ESP32

1. ESP-IDF y GPIO

ESP-IDF

ESP-IDF (*Espressif IoT Development Framework*) es el entorno de desarrollo oficial de Espressif para programar los microcontroladores ESP32.

Incluye:

- Herramientas para compilar, cargar y depurar programas.
- Bibliotecas y componentes para utilizar los periféricos del ESP32.
- Funciones para controlar GPIO, comunicaciones, Wi-Fi, Bluetooth y otros recursos.

GPIO

GPIO significa *General Purpose Input/Output* (entrada/salida de propósito general). Permite configurar los pines del microcontrolador como entradas o salidas.

Algunos registros del ESP32 permiten modificar directamente el estado de los pines:

- **W1TS (Write One to Set):** escribir `1` activa el bit correspondiente; escribir `0` lo mantiene sin cambios.
- **W1TC (Write One to Clear):** escribir `1` desactiva el bit correspondiente; escribir `0` lo mantiene sin cambios.
- **GPIO_IN:** permite leer el estado lógico de los pines de entrada.
- **GPIO_OUT:** representa el estado de salida de los GPIO.

Estos registros permiten controlar los pines mediante operaciones sobre bits, sin modificar innecesariamente los demás.

2. Pines del ESP32

No todos los pines de la placa tienen la misma función. Algunos pueden utilizarse como GPIO, mientras que otros están destinados a alimentación, comunicación o configuración de arranque.

2.1. Pines de uso general

Son los pines que pueden utilizarse para conectar componentes externos, por ejemplo:

- LEDs.
- Pulsadores.

- Sensores.
- Otros circuitos electrónicos.

Algunos pines también cumplen funciones especiales durante el arranque del microcontrolador.

2.2. Pines de configuración de arranque (Strapping Pins)

Son pines cuyo estado lógico se lee cuando el ESP32 se enciende o reinicia. Según esos niveles, el chip selecciona determinadas opciones de funcionamiento.

Por ejemplo:

- **GPIO0:** si está en nivel bajo durante el arranque, permite entrar en el modo de descarga del programa. Se relaciona con el botón BOOT.
- **GPIO2:** participa en la configuración de arranque y debe respetar las condiciones indicadas para el modo de descarga.
- **GPIO12:** selecciona el voltaje de alimentación de la memoria Flash. Si está en alto durante el arranque, puede seleccionar 1.8 V; en bajo selecciona 3.3 V en la configuración habitual del ESP32 clásico.

Por eso, no se deben conectar componentes que fuercen niveles incorrectos en estos pines durante el arranque.

2.3. Pines de solo entrada

Algunos GPIO del ESP32 clásico, como GPIO34, GPIO35, GPIO36 y GPIO39, funcionan únicamente como entradas.

Características:

- Permiten leer señales externas.
- No pueden utilizarse como salidas.
- No disponen de resistencias internas pull-up o pull-down.

2.4. Comunicación serial

La comunicación serial permite transmitir información entre el ESP32 y otros dispositivos mediante señales eléctricas.

Utiliza dos líneas principales:

- **TX (Transmit):** transmite información.
- **RX (Receive):** recibe información.

Son dos líneas de comunicación que permiten intercambiar datos. En una conexión entre dos dispositivos, normalmente TX de uno se conecta a RX del otro.

También se utiliza la comunicación serial para cargar programas y observar mensajes de diagnóstico desde el monitor serie.

2.5. Pines de alimentación y reinicio

Estos pines no son GPIO de propósito general.

Pin	Función
GND	Tierra o referencia de 0 V del circuito.
3V3	Alimentación de 3.3 V. En las placas habituales puede proporcionar energía a circuitos externos dentro de los límites permitidos.
VIN	Entrada de alimentación de la placa. En muchas DevKit V1 se utiliza para recibir 5 V, aunque depende del diseño de la placa.
EN	Permite habilitar o reiniciar el ESP32.

Importante: no se debe confundir VIN con una salida fija de 2.5 V. En una DevKit V1 habitual, VIN está asociado a la alimentación de 5 V. Siempre hay que comprobar el diseño de la placa antes de conectar una fuente externa.

3. Programa de computadora y programa embebido

Existen diferencias entre un programa que se ejecuta en una computadora y uno que funciona dentro de un microcontrolador.

Programa de computadora	Programa embebido
Es iniciado por el sistema operativo.	Comienza a ejecutarse cuando el dispositivo recibe alimentación y completa el arranque.
El sistema operativo administra su ejecución.	Controla directamente el hardware mediante los recursos del microcontrolador y su entorno de ejecución.
Normalmente realiza una tarea y termina.	Generalmente permanece funcionando continuamente.
Cuando termina, devuelve el control al sistema operativo.	Puede seguir ejecutándose mientras el dispositivo tenga alimentación.

En un microcontrolador, el programa normalmente no tiene un final como el de una aplicación de computadora. Permanece ejecutándose para supervisar entradas, controlar salidas y responder a eventos.

Por eso, se puede comparar con un guardia que permanece atento y realiza diferentes tareas durante todo su turno.

4. Secuencia de arranque y ejecución del ESP32

Cuando se conecta la alimentación al ESP32, el microcontrolador sigue una secuencia antes de comenzar a ejecutar el programa del usuario.

4.1. ROM Bootloader

Es el código de arranque que viene integrado de fábrica en el chip y se encuentra almacenado en la memoria ROM.

Sus funciones principales son:

- Ejecutar las instrucciones iniciales de arranque.
- Leer los pines de configuración (*strapping pins*).
- Determinar el modo de arranque.
- Iniciar el proceso de carga del siguiente programa.

4.2. Bootloader de Flash

Después de ejecutar el código de la ROM, se carga el segundo nivel del bootloader, almacenado en la memoria Flash.

Sus funciones incluyen:

- Revisar la información de las particiones de la memoria Flash.
- Localizar la aplicación que debe ejecutarse.
- Preparar el proceso para iniciar el programa principal.

4.3. Aplicación principal: `app_main()`

En ESP-IDF, la función principal de la aplicación se llama `app_main()`.

```
void app_main(void)
{
    // Código principal de la aplicación
}
```

Esta función se ejecuta después del proceso de arranque y permite iniciar las tareas del programa.

ESP-IDF utiliza FreeRTOS para administrar tareas, prioridades y recursos del procesador.

4.4. FreeRTOS y las tareas

FreeRTOS es un sistema operativo en tiempo real que permite ejecutar y organizar diferentes tareas dentro del microcontrolador.

Una tarea puede encargarse de:

- Leer sensores.
- Controlar LEDs.
- Recibir información por comunicación serial.
- Procesar datos.

En el ESP32 clásico, que tiene dos núcleos, las tareas pueden distribuirse entre ellos. La asignación depende de la configuración del sistema.

4.5. `setup()` y `loop()` en Arduino

Cuando se programa el ESP32 utilizando el entorno Arduino, se utilizan dos funciones principales:

`**setup()**`

Se ejecuta una sola vez al iniciar el programa.

Se utiliza para:

- Configurar los pines.
- Inicializar la comunicación serial.
- Configurar sensores y periféricos.
- Preparar las variables y recursos necesarios.

```
void setup() {
    Serial.begin(115200);
    pinMode(2, OUTPUT);
}
```

`**loop()**`

Se ejecuta repetidamente mientras el microcontrolador permanece funcionando.

Se utiliza para:

- Leer entradas.
- Procesar información.
- Controlar salidas.
- Ejecutar las acciones principales del programa.

```
void loop() {
    digitalWrite(2, HIGH);
    delay(1000);

    digitalWrite(2, LOW);
```

```
    delay(1000);
}
```

La función `loop()` vuelve a ejecutarse después de terminar cada ciclo. Por eso, normalmente no se utiliza para finalizar el programa, sino para mantener su funcionamiento.

Relación entre ESP-IDF y Arduino: ESP-IDF es el framework oficial de Espressif. Arduino-ESP32 es otro entorno de programación construido sobre ESP-IDF. En Arduino se trabaja principalmente con `setup()` y `loop()`, mientras que en ESP-IDF se utiliza `app_main()` y se pueden crear tareas de FreeRTOS directamente.

5. Sketch y funciones de programación

Un *sketch* es el nombre que recibe el archivo o programa desarrollado en el entorno Arduino.

En el ESP32 se puede utilizar el framework Arduino o ESP-IDF para desarrollar aplicaciones.

Comunicación serial: `Serial.begin()`

Permite iniciar la comunicación serial y establecer la velocidad de transmisión de datos.

```
Serial.begin(115200);
```

El valor `115200` representa la velocidad de transmisión en baudios. El monitor serial debe configurarse con la misma velocidad para mostrar correctamente los mensajes.

Variables estáticas: `static`

La palabra clave `static` permite conservar el valor de una variable durante toda la ejecución del programa.

Cuando se declara dentro de una función, la variable conserva su valor entre llamadas a esa función.

```
void loop() {
    static int contador = 0;

    contador++;
    Serial.println(contador);
    delay(1000);
}
```

Aunque `loop()` se ejecute repetidamente, `contador` no vuelve a cero en cada ejecución.

Consumo de energía y Deep Sleep

El modo *Deep Sleep* permite reducir considerablemente el consumo de energía del ESP32 cuando no necesita estar realizando operaciones.

Se utiliza en dispositivos que funcionan con baterías o que deben permanecer en espera durante largos períodos.

Puede ser útil cuando un programa no necesita ejecutar continuamente su ciclo principal, sino que puede dormir y despertarse posteriormente mediante una fuente de activación configurada.

6. Tipos de datos enteros

En programación de microcontroladores se utilizan tipos de datos que permiten almacenar números dentro de un rango determinado.

Dos tipos comunes son `uint8_t` e `int8_t`.

Tipo de dato	Significado	Rango	Tamaño
`uint8_t`	Entero de 8 bits sin signo.	0 a 255	1 byte
`int8_t`	Entero de 8 bits con signo.	-128 a 127	1 byte

La diferencia está en cómo se interpreta el bit más significativo.

- `uint8_t`: todos los bits se utilizan para representar valores positivos y cero.
- `int8_t`: utiliza una representación con signo, por lo que también permite almacenar valores negativos.

Ejemplo

```
uint8_t edad = 25;
int8_t temperatura = -10;
```

Ambas variables ocupan un byte, pero permiten almacenar diferentes rangos de valores.

7. Desbordamiento (Overflow)

El desbordamiento ocurre cuando una variable intenta almacenar un valor que supera el límite permitido por su tipo de dato.

En una variable de 8 bits sin signo (`uint8_t`), el rango es de 0 a 255.

Por ejemplo:

```
uint8_t numero = 255;
numero++;
```

El resultado será `0`, porque al superar el máximo permitido el valor vuelve al inicio del rango.

En una variable `int8_t`, el comportamiento también es diferente porque trabaja con números con signo. No se debe asumir que superar el rango produce un resultado válido: el desbordamiento de enteros con signo en C/C++ tiene comportamiento indefinido.

Por eso, es importante conocer los límites de cada tipo de dato y evitar operaciones que los excedan.

8. Función `delay()`

La función `delay()` permite detener temporalmente la ejecución del programa durante un tiempo determinado.

```
delay(1000);
```

El valor se expresa en milisegundos:

- `delay(1000)`: espera 1 segundo.
- `delay(500)`: espera medio segundo.
- `delay(100)`: espera 100 milisegundos.

Durante una espera de este tipo, el código de la tarea que la ejecuta queda detenido temporalmente. En el ESP32, el planificador de FreeRTOS puede permitir que otras tareas continúen ejecutándose.

9. Resistencia pull-up y estados de un pulsador

La resistencia pull-up mantiene la entrada en nivel lógico alto cuando el pulsador está suelto.

Cuando el pulsador conecta el pin a GND, la entrada pasa a nivel bajo.

Estado del pulsador	Nivel lógico	Lectura
Suelto	HIGH	`1`
Pulsado	LOW	`0`

Este comportamiento se conoce como lógica activa en bajo (*active-low*).

En el ESP32 se puede activar una resistencia pull-up interna mediante:

```
pinMode(4, INPUT_PULLUP);
```

En este caso, el programa leerá `HIGH` cuando el botón esté suelto y `LOW` cuando esté pulsado.